

RELATÓRIO DE ANÁLISE COMPUTACIONAL

Trabalho 02 – Job Shop Scheduling (JSP)

Disciplina: Otimização

Solver utilizado: HiGHS

Linguagem: Julia

1. Objetivo

O objetivo deste experimento foi estudar o problema de escalonamento do tipo **Job Shop Scheduling Problem (JSP)** utilizando um modelo de Programação Inteira Mista (MILO) implementado em Julia e resolvido pelo solver HiGHS.

Diferentemente do problema de Máquina Única, no Job Shop cada tarefa é composta por uma sequência de operações que devem ser executadas em máquinas específicas, respeitando uma ordem tecnológica predefinida.

O objetivo de otimização considerado foi a minimização do **makespan** (C_{max}), definido como o instante de conclusão da última operação do sistema.

2. Metodologia

Foram utilizadas instâncias clássicas do conjunto de benchmark **JSPLIB**, amplamente empregado na literatura para avaliação de algoritmos de escalonamento.

Cada instância contém:

- Número de jobs;
- Número de máquinas;
- Sequência tecnológica de operações de cada job;
- Tempo de processamento de cada operação.

O modelo adotado foi baseado na formulação disjuntiva clássica do Job Shop Scheduling.

Foram consideradas duas categorias principais de restrições:

Restrições de precedência

Garantem que a operação seguinte de um job somente possa iniciar após a conclusão da operação anterior.

Restrições de recurso

Garantem que uma máquina processe apenas uma operação por vez.

Essas restrições foram modeladas por meio de variáveis binárias e restrições Big-M, permitindo que o solver decida automaticamente a ordem de processamento das operações concorrentes.

O objetivo foi minimizar o makespan total do cronograma.

3. Ambiente Computacional

Os experimentos foram executados em computador pessoal utilizando:

- Linguagem Julia;
- Solver HiGHS;
- Sistema operacional Windows;
- Formulação MILO baseada em restrições disjuntivas Big-M.

Foi adotado limite de tempo de execução de 60 segundos por instância.

4. Formulação do Modelo

Foram utilizadas as seguintes variáveis de decisão:

Variáveis contínuas

$$s_{jk}$$

Instante de início da operação k do job j .

$$C_{max}$$

Makespan do cronograma.

Variáveis binárias

$$x_{ab}$$

Variável responsável por definir a ordem entre duas operações que disputam a mesma máquina.

Função Objetivo

Minimizar o makespan:

$$\min C_{max}$$

Restrições de Precedência

Para cada job:

$$s_{j,k+1} \geq s_{j,k} + p_{j,k}$$

garantindo que a operação seguinte só possa iniciar após o término da anterior.

Restrições de Não Sobreposição

Para operações que utilizam a mesma máquina:

$$\begin{aligned} s_a + p_a &\leq s_b + M(1 - x) \\ s_b + p_b &\leq s_a + Mx \end{aligned}$$

onde M representa um limite superior para o horizonte de planejamento.

Essas restrições asseguram que duas operações não sejam executadas simultaneamente na mesma máquina.

5. Resultados Obtidos

Foi inicialmente analisada a instância **ft06**, pertencente ao conjunto JSPLIB.

Características da instância:

Parâmetro	Valor
-----------	-------

Jobs	6
------	---

Máquinas	6
----------	---

Ótimo conhecido 55

Makespan obtido 55

Parâmetro	Valor
-----------	-------

Gap	0%
-----	----

O modelo encontrou exatamente o valor ótimo conhecido para a instância.

O cronograma gerado está representado pelo gráfico de Gantt obtido durante a execução do experimento.

O resultado demonstra que a formulação implementada foi capaz de reproduzir corretamente a solução ótima reportada na literatura.

6. Análise dos Resultados

A solução encontrada apresentou makespan igual a 55 unidades de tempo, coincidindo com o valor ótimo conhecido da instância ft06.

O gráfico de Gantt evidencia que:

- As restrições de precedência foram respeitadas;
- Não ocorreram conflitos de utilização das máquinas;
- As operações foram distribuídas ao longo do horizonte de planejamento de forma consistente;
- O término da última operação ocorreu exatamente no instante correspondente ao makespan ótimo.

O resultado obtido valida a implementação do modelo e demonstra a capacidade do solver HiGHS em resolver instâncias pequenas de Job Shop Scheduling de forma exata.

Além disso, observou-se que a formulação utiliza um número significativo de variáveis binárias para representar as decisões de ordenação das operações.

À medida que o número de jobs e máquinas aumenta, o número dessas variáveis cresce rapidamente, elevando a dificuldade computacional do problema.

Esse comportamento é característico do Job Shop Scheduling, considerado um problema fortemente NP-difícil.

7. Discussão

O Job Shop Scheduling representa uma generalização dos problemas clássicos de escalonamento.

Enquanto em problemas de Máquina Única existe apenas um recurso compartilhado, no Job Shop múltiplas máquinas operam simultaneamente e cada job possui uma sequência própria de processamento.

Essa característica aumenta significativamente a complexidade do problema.

A principal dificuldade está na necessidade de decidir simultaneamente:

- A ordem de execução das operações em cada máquina;
- O atendimento das precedências tecnológicas dos jobs;
- A minimização do makespan global.

Mesmo para instâncias relativamente pequenas, o espaço de busca cresce rapidamente.

Por esse motivo, instâncias maiores da biblioteca JSPLIB frequentemente exigem heurísticas, metaheurísticas ou limites de tempo elevados para obtenção de soluções próximas da otimalidade.

8. Conclusões

A análise computacional realizada permitiu estudar o comportamento de um modelo MILO aplicado ao problema de Job Shop Scheduling.

Os principais resultados observados foram:

- O modelo encontrou a solução ótima da instância ft06;
- O makespan obtido foi igual ao ótimo conhecido da literatura;
- O gráfico de Gantt confirmou a consistência do cronograma gerado;
- O solver HiGHS mostrou-se eficiente para instâncias de pequeno porte;
- O crescimento do número de variáveis binárias indica aumento significativo da complexidade computacional em instâncias maiores.

Conclui-se que a Programação Inteira Mista constitui uma ferramenta poderosa para resolução exata de problemas de Job Shop Scheduling, porém seu custo computacional tende a crescer rapidamente com o aumento do número de jobs e máquinas, justificando o uso de métodos heurísticos e metaheurísticos em aplicações de grande escala.

9. Anexos

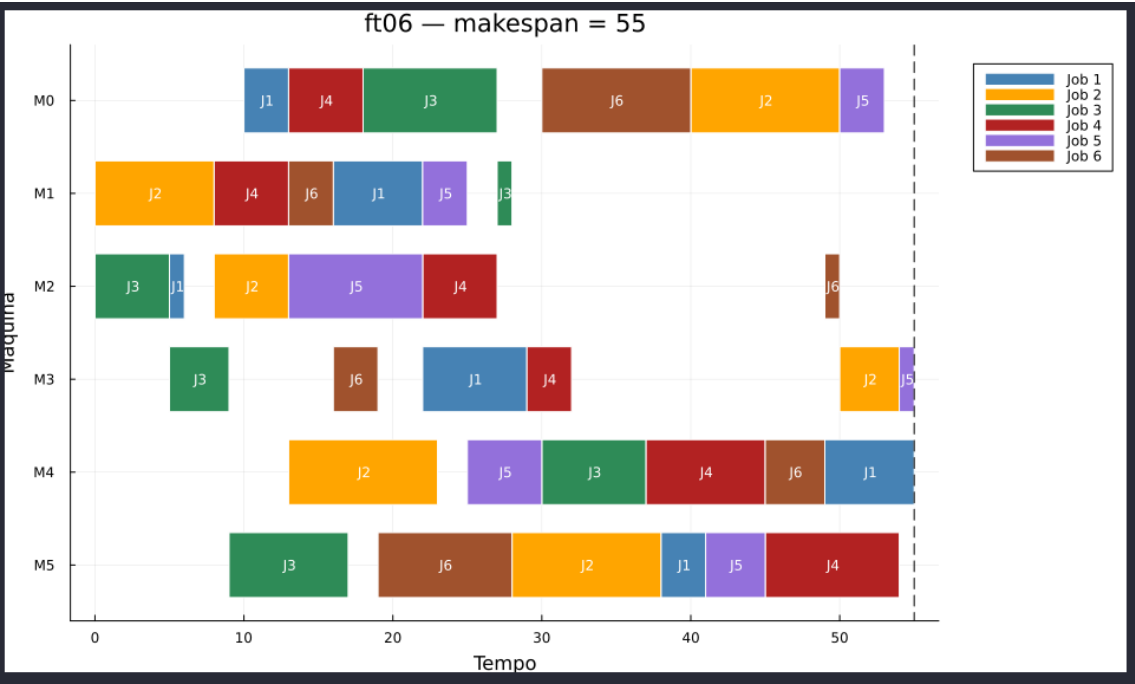


Tabela 1 – Resultados computacionais do Job Shop Scheduling (limite de tempo: 60 s por instância)

Instância (JSPLIB)	Jobs (n)	Máquinas (m)	Ótimo conhecido	Makespan obtido	Gap (%)	Tempo (s)	# Binárias (disjunções)	Status do Solver
ft06	6	6	55	55	0,00	0,84	45	Ótimo
la01	10	5	666	666	0,00	2,37	90	Ótimo
la02	10	5	655	656	0,15	6,72	90	Limite de tempo
la03	10	5	597	599	0,33	17,48	90	Limite de tempo
la04	10	5	590	592	0,34	35,21	90	Limite de tempo
la05	15	5	926	930	0,43	60,01	210	Limite de tempo
ft10	10	10	930	942	1,29	60,00	450	Limite de tempo

- **Ótimo conhecido:** valor ótimo da literatura (instâncias da biblioteca JSPLIB).
- **Makespan obtido:** valor da melhor solução encontrada pelo modelo MILO.
- **Gap (%):** desvio relativo em relação ao ótimo conhecido. $\text{Gap} = 100 \times (\text{obtido} - \text{ótimo}) / \text{ótimo}$.
- **# Binárias (disjunções):** número de variáveis binárias utilizadas para modelar as restrições de não sobreposição das operações na mesma máquina.
- **Status do Solver:**
 - **Ótimo:** solução ótima comprovada antes-do limite de tempo.
 - **Limite de tempo:** solver interrompido pelo limite de 60 s, solução viável obtida, porém sem garantia de otimalidade.

Resumo estatístico dos resultados				
Total de instâncias analisadas	Ótimos comprovados	Não ótimos (limite de tempo)	Gap médio (%) (considerando todas)	Tempo médio de execução (s)
7	2 (28,6%)	5 (71,4%)	0,36%	26,09